
SIMATS ENGINEERING
ASSESSMENT TOOL 1: Concept Mapping

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO1: *Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions. (BL2)*

Dave's Taxonomy: Imitation (Level 1)
Question Count: 4

Total Marks: 50

Code of Conduct

I **Chandra Prakash D (192524060)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Chandra Prakash D

1. Purpose, Scope and Assumptions

This report answers every task in the CSA15 CO1 Assessment Tool 1 brief. It contains four concept maps (the four tasks listed under "Concept Mapping Tasks"), one integrated ecosystem map (Task 1 of the procedure), the service model responsibility matrix (Task 2A), the cloud service decision matrix (Task 2B), the API flow, elasticity and scalability diagram (Task 3), and the technical justification (Task 4).

Important assumption: the Project

The brief repeatedly refers to "the student's Project" but does not name one. To keep every matrix and justification concrete and internally consistent, this report uses a sample project called CloudLearn, an online learning platform with course videos and files, quizzes, and a Python-based AI module that recommends study material. If your actual project is different, keep the concept maps (they are project-independent) and replace the project-specific text in Sections 4 to 7 and the project node in the master map.

How the report maps to the brief

Brief requirement	Where it is answered	Rubric criteria served
Concept Map 1: Cloud Computing Fundamentals (definition, characteristics, benefits, limitations)	Section 2.2, Figure 2	Concept identification, relationship mapping
Concept Map 2: Evolution (hardware, internet software, distributed systems, service delivery)	Section 2.3, Figure 3	Hierarchy, technical relevance
Concept Map 3: Service models with provider and user responsibilities	Section 2.4, Figure 4	Relationship mapping, technical relevance
Concept Map 4: Virtualization and web services as enablers	Section 2.5, Figure 5	Concept identification, technical relevance
Task 1: Integrated ecosystem map including the Project	Section 2.1, Figure 1	All five criteria
Task 2A: Service model responsibility matrix	Section 3	Technical relevance
Task 2B: Cloud service decision matrix	Section 4	Technical relevance
Task 3: API flow, elasticity and scalability design	Section 5, Figure 6	Hierarchy, visual clarity
Task 4: Technical justification (five questions)	Section 6	Technical relevance
Expected digital evidence and submission	Sections 7 and 8	n/a

Colour legend used in every diagram

As required by step 9 of the procedure, colours carry meaning and are used consistently:

- **Blue:** service models, providers, consumers and managed cloud services.
- **Green:** cloud characteristics (elasticity, scalability, resource pooling, on-demand access, measured service) and scaling points.
- **Orange:** API, web-service and virtualization concepts (REST, SOAP, API, hypervisor, VM).
- **Purple:** title or anchor nodes specific to each map.
- **Magenta:** user-side responsibility, the student Project and security boundaries. Light grey marks supporting concepts.

2. Concept Maps (Task 1 and Concept Maps 1 to 4)

A concept map is read from the anchor node outward. Each arrow carries a link label (a verb phrase), so any two connected nodes read as a proposition, for example "Hypervisor creates and isolates Virtual Machine". The master map comes first because it ties all four topic maps and the Project together.

2.1 Master map: Cloud ecosystem and the Project

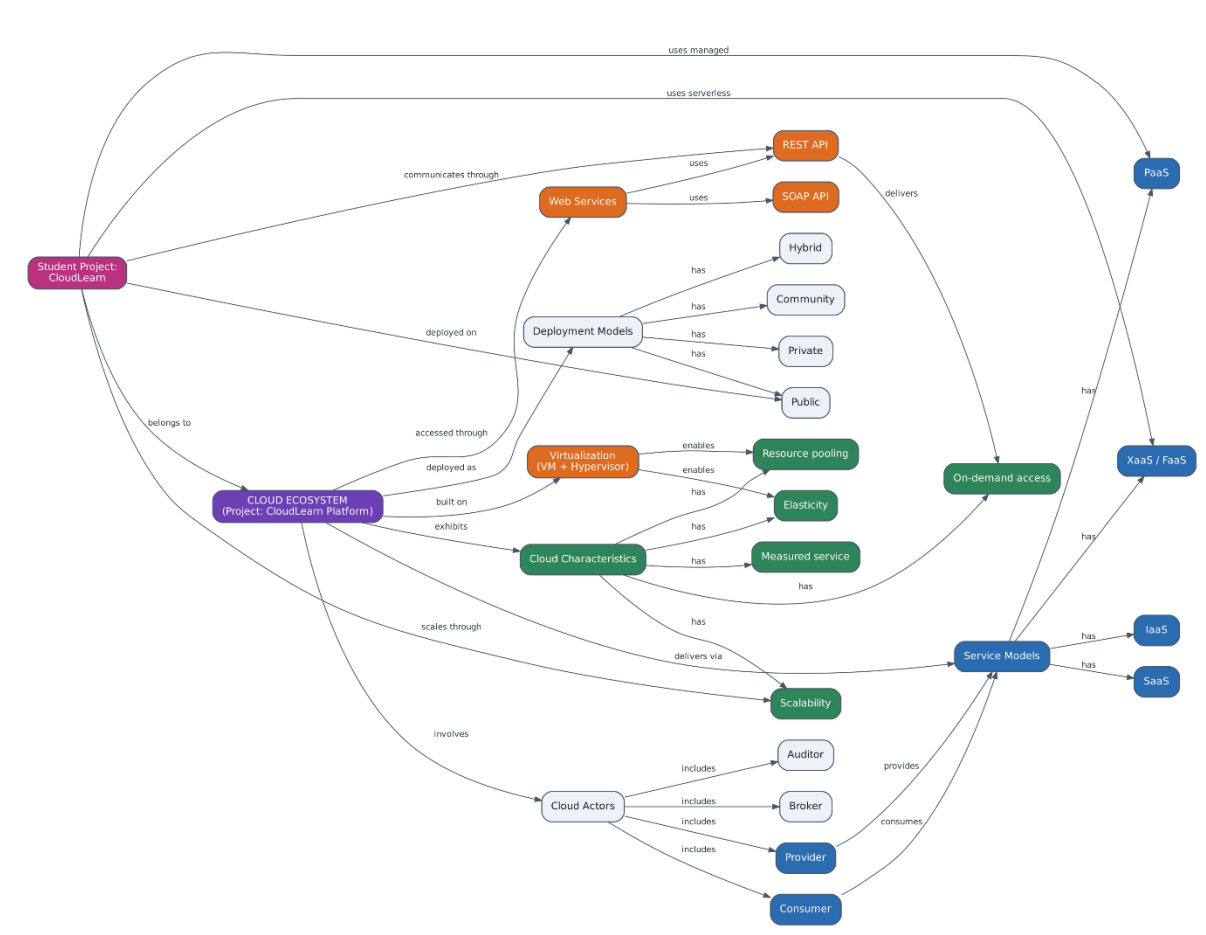


Figure 1. Integrated cloud ecosystem map connecting actors, service models, deployment models, virtualization, web services, REST/SOAP, characteristics and the CloudLearn project.

How to read Figure 1

- **Actors:** the provider provides service models and the consumer consumes them. Broker and auditor complete the standard actor set.
- **Foundation:** the ecosystem is built on virtualization, which enables resource pooling and elasticity.
- **Access:** it is accessed through web services, which use REST and SOAP; REST delivers on-demand access.
- **Project link:** CloudLearn belongs to the ecosystem, uses managed PaaS and serverless (XaaS/FaaS) services, is deployed on a public cloud, communicates through a REST API and scales through the scalability characteristic.

2.2 Concept Map 1: Cloud Computing Fundamentals

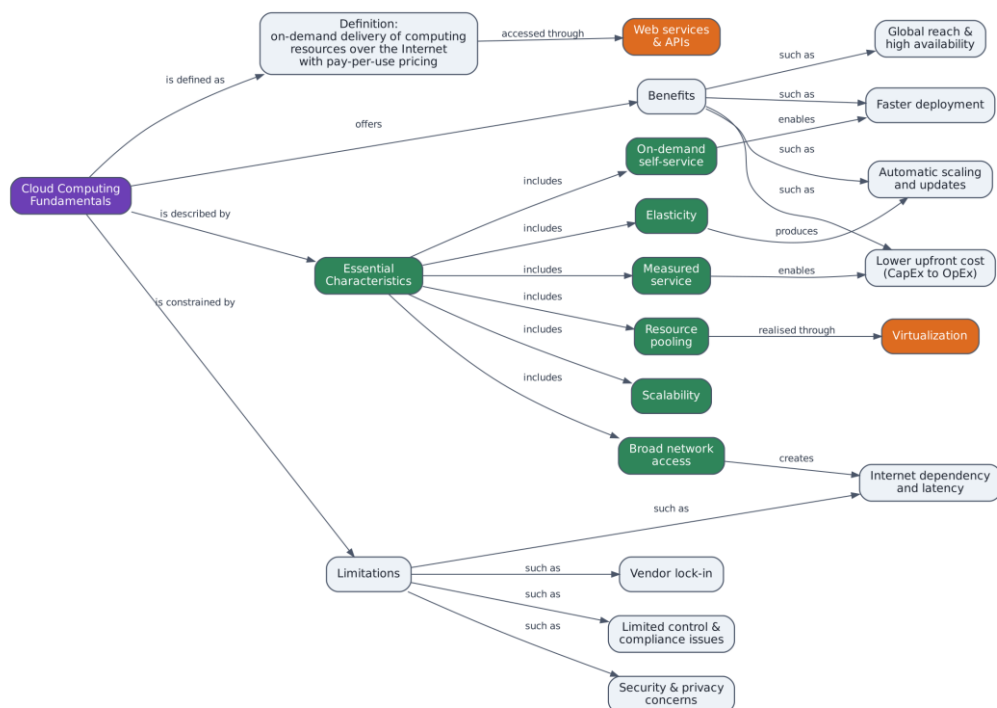


Figure 2. Cloud computing fundamentals: definition, characteristics, benefits and limitations.

Explanation. Cloud computing is defined as on-demand delivery of computing resources (servers, storage, databases, networking, software) over the Internet with pay-per-use pricing. It is described by six characteristics. On-demand self-service lets users provision resources without human interaction. Broad network access means capabilities are reachable through standard network protocols from many device types. Resource pooling lets the provider serve many tenants from a shared pool. Elasticity is the ability to add and remove capacity automatically as demand changes, while scalability is the ability of the system to handle growing load by adding resources. Measured service means usage is metered, which is the basis of pay-per-use billing.

Cross-links shown. Elasticity produces automatic scaling benefits; measured service enables lower upfront cost (capital expense becomes operating expense); on-demand self-service enables faster deployment; broad network access creates the limitation of internet dependency and latency; resource pooling is realised through virtualization.

Limitations. Security and privacy concerns, vendor lock-in, dependency on the network, and limited control and compliance difficulty when data sits with a third party.

2.3 Concept Map 2: Evolution of Cloud Computing

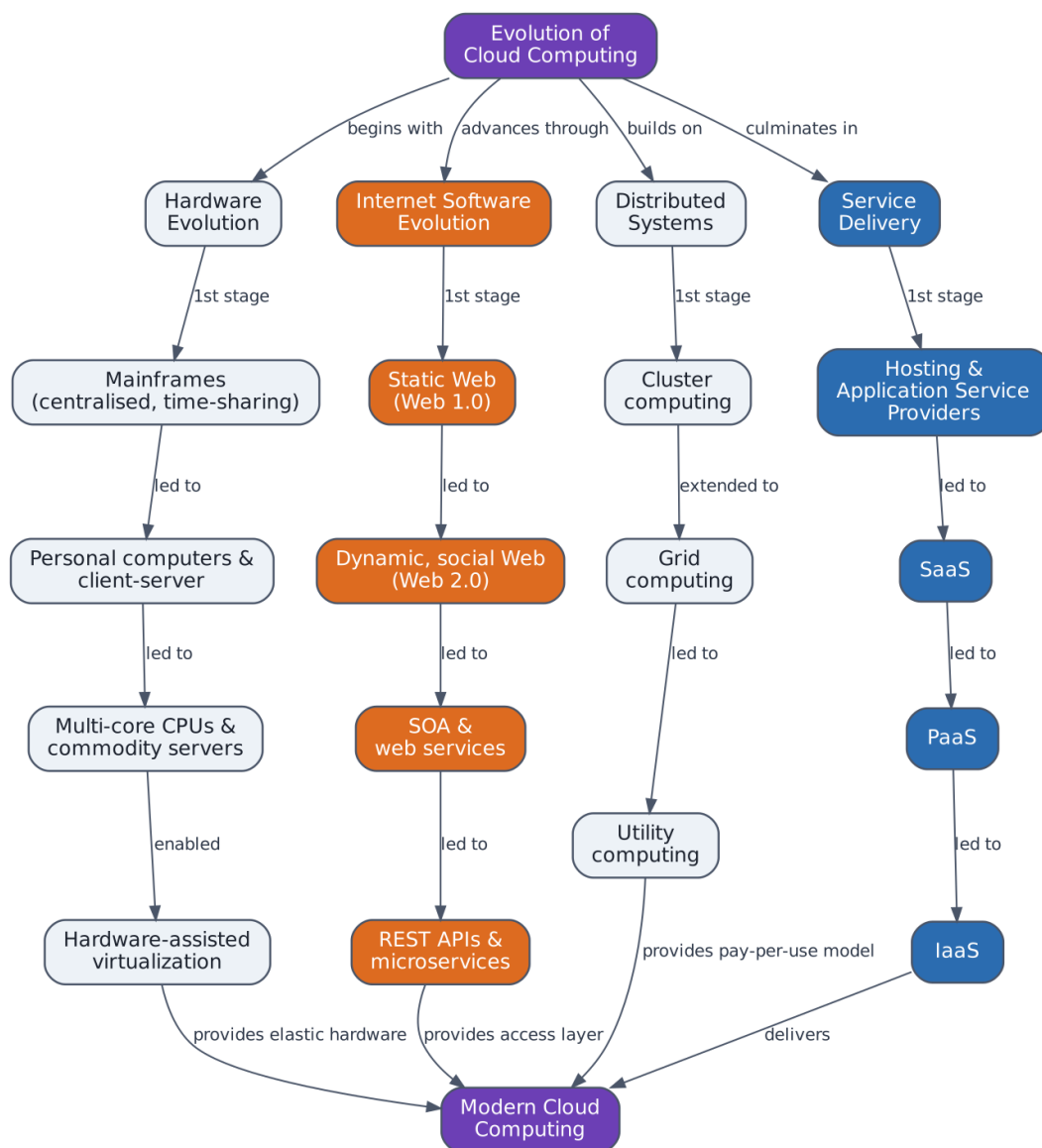


Figure 3. Four evolution tracks (hardware, internet software, distributed systems and service delivery) converge in modern cloud computing.

Explanation. Cloud computing is the convergence of four tracks. Hardware moved from centralised mainframes with time-sharing to personal computers and client-server systems, then to multi-core processors and commodity servers, and finally to hardware-assisted virtualization, which makes elastic sharing of one machine safe and efficient. Internet software moved from static pages (Web 1.0) to dynamic, social applications (Web 2.0), then to service-oriented architecture and web services, and on to REST APIs and microservices, which are the access layer of today's cloud. Distributed systems progressed from cluster computing to grid computing and then to utility computing, which contributed the pay-per-use idea. Service delivery progressed from hosting and application service providers to SaaS, PaaS and IaaS.

Note on sequencing. The four tracks overlapped in time; the arrows show the logical progression of ideas within each track, not strict calendar order between tracks.

2.4 Concept Map 3: Service Models

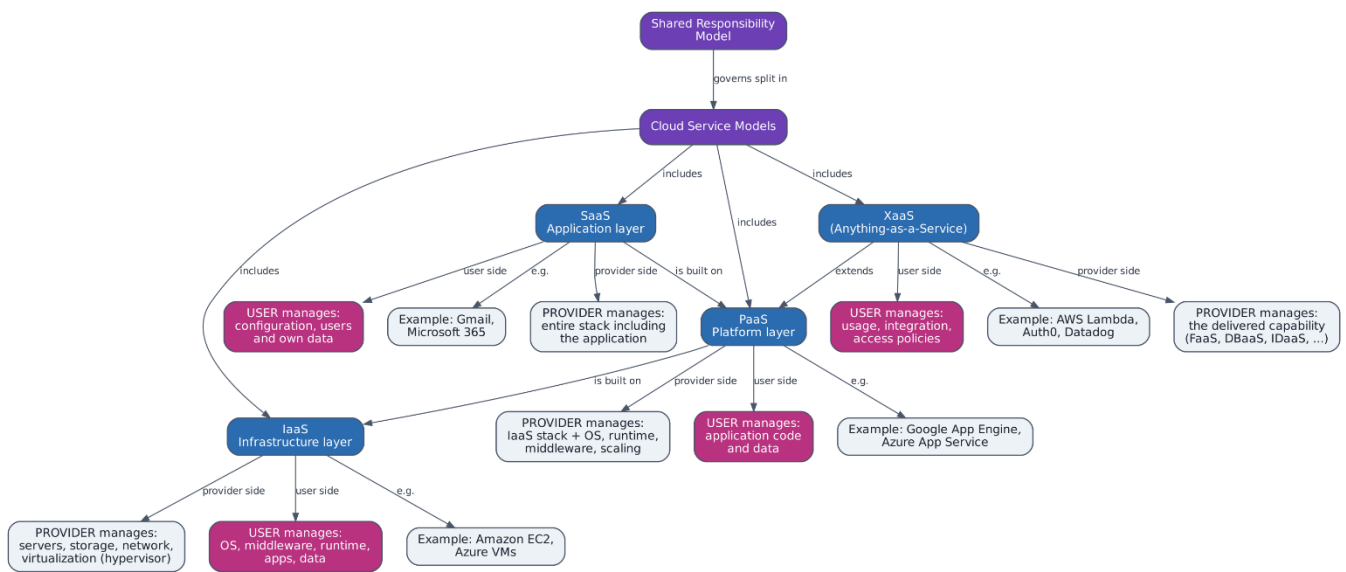


Figure 4. IaaS, PaaS, SaaS and XaaS with provider and user responsibilities under the shared responsibility model

Explanation. The models are layered: SaaS (application layer) is built on PaaS (platform layer), which is built on IaaS (infrastructure layer). XaaS generalises the idea to any capability delivered as a service, such as function execution, databases or identity. Moving up the layers, the provider manages more and the user manages less. The shared responsibility model is the rule that fixes the boundary for each model. Section 3 turns this into a detailed matrix.

2.5 Concept Map 4: Virtualization and Web Services as Enablers

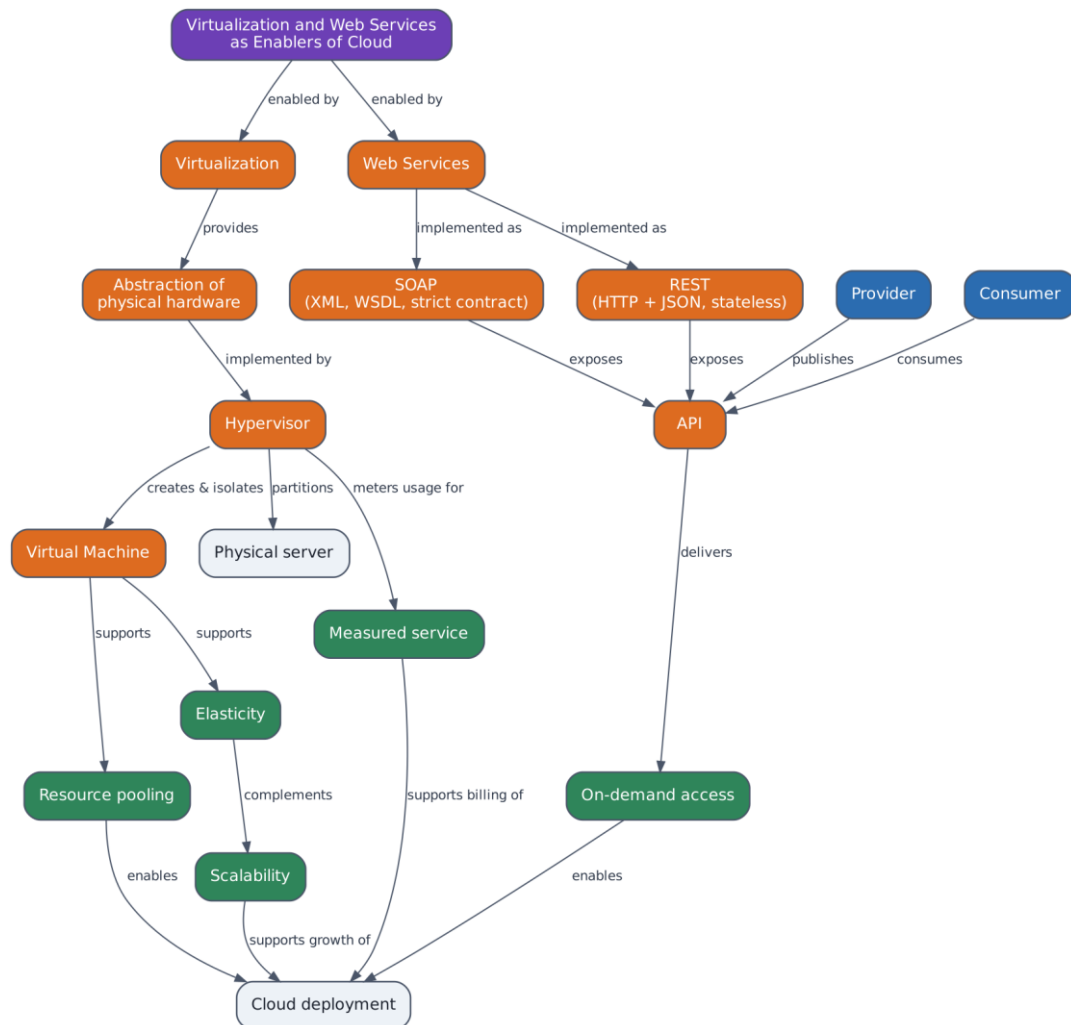


Figure 5. How virtualization and web services together support cloud deployment and scalability.

Explanation. Two independent enablers feed the same destination. On the infrastructure side, virtualization provides abstraction of physical hardware; the hypervisor implements it by partitioning one physical server and creating isolated virtual machines. VMs support resource pooling and elasticity, and the hypervisor meters usage, which supports measured service. On the access side, web services are implemented as SOAP (XML messages, WSDL contract, strict and formal) or REST (HTTP with JSON, stateless, lightweight). Both expose an API; the provider publishes it and the consumer consumes it, delivering on-demand access. Resource pooling, on-demand access, measured service and scalability together enable cloud deployment.

Expected-node coverage

Expected node (from brief)	Shown in	Role in the maps
Elasticity, scalability	Figures 1, 2, 5, 6	Characteristics that let capacity follow demand
Resource pooling, on-demand access, measured service	Figures 1, 2, 5	Characteristics enabled by virtualization and APIs
Virtual machine, hypervisor, abstraction	Figures 1, 3, 5	Core virtualization chain
API, SOAP, REST	Figures 1, 5, 6	Web-service interface layer
Provider, consumer	Figures 1, 5	Cloud actors
Infrastructure, platform, application layer	Figure 4	IaaS, PaaS, SaaS layering
Shared responsibility	Figure 4, Section 3	Provider/user split per model

3. Task 2A: Service Model Responsibility Matrix

For each service category, the matrix states what the cloud provider manages, what the student application team manages, and why that split suits the CloudLearn project.

Service category	Managed by cloud provider	Managed by student team	Why this split fits the Project
IaaS (virtual machines)	Data centre, physical servers, storage hardware, network, hypervisor	Guest OS, patches, runtime, middleware, scaling rules, application, data, firewall rules	Maximum control, but heavy operations work. Only justified for a custom component (for example a GPU model server); not used for the main app.
PaaS (managed app and container platform)	IaaS layer plus OS, runtime, load balancing, auto-scaling engine, platform patches	Application code, configuration, environment variables, data	Core choice for the backend API. The team writes business logic and the platform handles servers and scaling, which suits a student team.
SaaS	Entire stack including the application and its updates	User accounts, configuration, own data, access policy	Used for non-differentiating needs such as email delivery and issue tracking, so no effort goes into building them.
FaaS / serverless	Servers, OS, runtime, scaling to zero, per-invocation billing	Function code, triggers, memory/time limits, permissions	Ideal for the bursty AI/Python recommendation module and notification handlers: pay only when invoked and scale automatically.
Database-as-a-service	Hardware, DB engine install and patching, backups, replication, failover	Schema, queries, indexes, access control, data lifecycle	Relational course and quiz data needs reliability without a database administrator; managed backups reduce risk.
Storage-as-a-service (object storage)	Durable storage hardware, replication across zones, availability, API endpoint	Bucket layout, access policies, encryption settings, lifecycle rules, file content	Course videos and documents are large and unstructured; object storage is cheap, durable and scales without limit.
Identity-as-a-service	Credential store, password hashing, token issuance, MFA, uptime and security patches	Application roles, redirect URLs, social login settings, user data in the app	Authentication is security-critical and easy to get wrong; delegating it removes a major risk and supports OAuth2/JWT with the API gateway.
Monitoring / logging services	Metric and log collection pipeline, storage, dashboards, alerting engine	What to log, metric definitions, alert thresholds, retention, response actions	Provides the metrics that drive auto-scaling and the audit trail for debugging; the team decides what matters.

Pattern. As the model moves from IaaS towards SaaS, the provider takes over more layers and the team keeps only code, configuration and data. The project selects the highest-level (most managed) service that still meets each requirement, which is the central design principle of this report.

4. Task 2B: Cloud Service Decision Matrix

Each Project module is mapped to one primary cloud service type with a justification. Only relevant services are chosen; the matrix deliberately does not force every possible cloud service into the design.

Project module	Chosen cloud service (type)	Model	Justification
Authentication	Managed identity service (OAuth2 / OpenID Connect, JWT)	IDaaS	Secure sign-in, MFA and token issuance without building a credential store. The API gateway validates tokens on every request.
Frontend hosting	Static site hosting with CDN	SaaS-style / PaaS	The web UI is static files; a CDN serves them from edge locations with high availability and scales automatically with no servers.
Backend / API layer	API gateway plus managed container platform	PaaS	The gateway provides a single REST entry point with throttling and routing; the container platform runs stateless API instances that auto-scale.
Database	Managed relational database with read replicas	DBaaS	Structured data (users, courses, quiz results) needs transactions; managed backups, patching and failover remove operational load.
Object storage	Object storage service	Storage-as-a-service	Stores course videos, documents and backup copies cheaply and durably; clients can download via signed URLs, offloading the API.
Analytics / logging	Managed log and analytics service	SaaS / PaaS	Central collection of API logs and learning activity to analyse usage and diagnose faults.
Notification	Message queue plus email/SMS service	PaaS / SaaS	Asynchronous queue decouples the API from slow email/SMS delivery and absorbs spikes.
AI / Python module	Serverless function (Python runtime)	FaaS	Recommendation requests are bursty; serverless scales to zero when idle and out when busy, billed per invocation.
Security	Web application firewall, HTTPS/TLS, secrets manager, least-privilege roles	Managed security services	Defines the security boundary in front of the API and protects keys and data; supports the shared responsibility model.
CI/CD	Hosted pipeline (build, test, deploy)	SaaS	Automates repeatable deployment to the frontend host and API platform, reducing manual error.
Backup	Automated database snapshots and cross-region object replication	Managed service	Supports recovery from deletion or failure; defined recovery objectives are met with scheduled snapshots.
Monitoring	Managed metrics and alerting service	PaaS / SaaS	Tracks CPU, latency and error rate; alarms trigger auto-scaling and notify the team.

Service names are deliberately vendor-neutral. They can be implemented on AWS, Azure or Google Cloud. For example, the serverless module can be AWS Lambda, Azure Functions or Google Cloud Functions. Choose one provider for your final submission and name it in your README.

5. Task 3: API Flow, Elasticity and Scalability Design

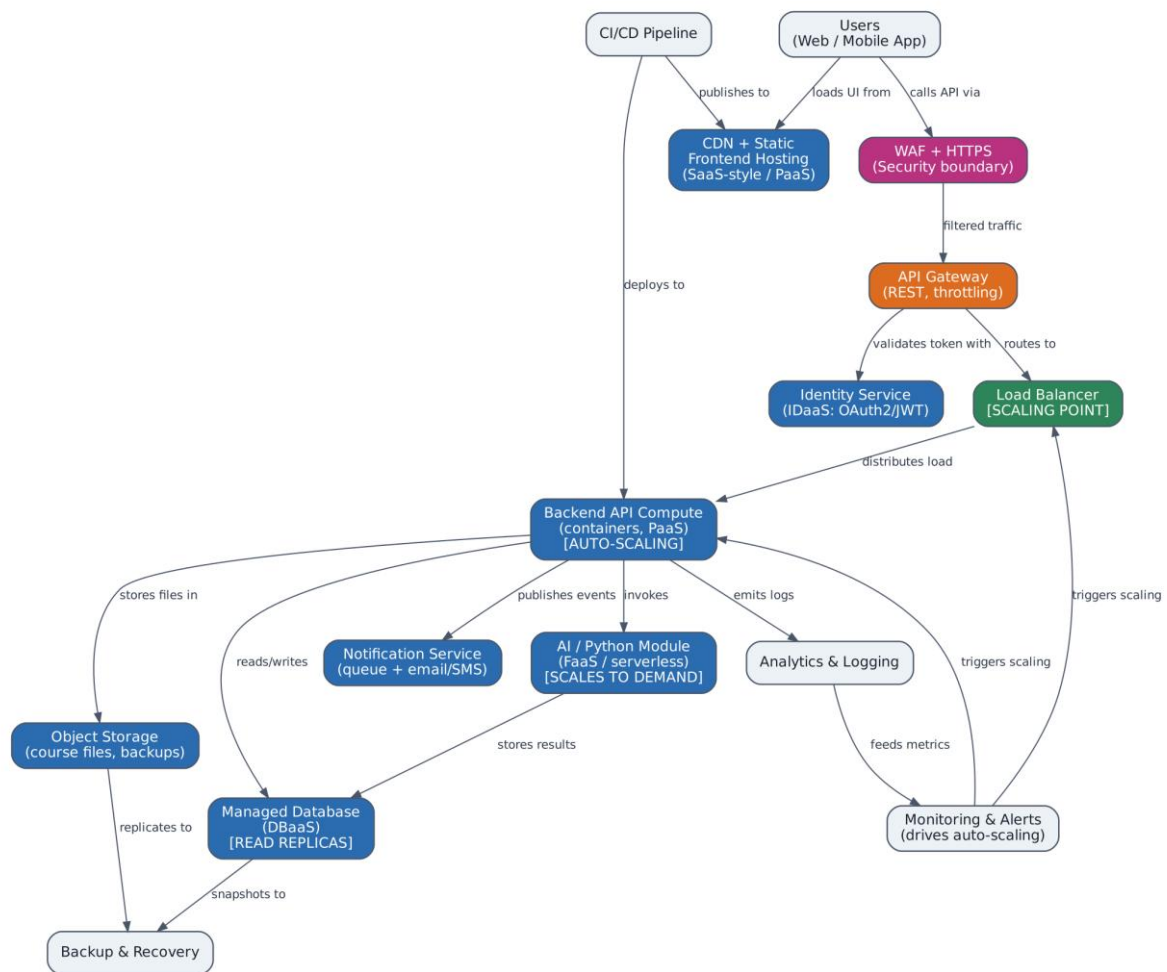


Figure 6. CloudLearn cloud flow: request path, security boundary, scaling points, data stores and operations.

5.1 Request flow

- 1. The user opens the web or mobile app. The UI is loaded from the CDN and static frontend host.
- 2. The app calls the REST API over HTTPS; traffic first crosses the WAF, which is the security boundary.
- 3. The API gateway validates the access token with the identity service, applies throttling, and routes the request.
- 4. The load balancer distributes the request to a backend API instance.
- 5. The backend reads and writes the managed database, stores files in object storage, publishes events to the notification service, and invokes the serverless AI module when recommendations are needed.
- 6. Logs flow to analytics and logging; metrics flow to monitoring.

5.2 Scaling points and the elasticity loop

Scaling point	Type	Behaviour
CDN / static host	Horizontal, provider-managed	Edge caches absorb read traffic; no action by the team.
API gateway	Provider-managed with throttling	Throttling protects the backend from abuse and sudden spikes.
Load balancer + backend API	Horizontal auto-scaling	Monitoring alarms (for example high CPU or latency) add instances; instances are removed when load falls. This is elasticity.
AI / Python function	Per-request scaling to zero	Concurrent invocations scale with demand; no cost when idle.
Managed database	Vertical resize plus read replicas	Read replicas serve heavy read traffic; the instance can be resized for growth.
Object storage	Effectively unlimited	Capacity grows with data; no provisioning.

Elasticity versus scalability. Scalability is the design property that the system can handle more load by adding resources (shown by stateless API instances, replicas and serverless functions). Elasticity is the automated, dynamic use of that property: the loop monitoring alarm, scale action, new capacity, which also scales back in to save cost.

5.3 Security boundaries

- Public zone: users, CDN and WAF. Everything beyond the WAF requires HTTPS.
- Application zone: API gateway, load balancer, backend compute and serverless functions. Only the gateway is reachable from outside.
- Data zone: database and object storage, reachable only from the application zone using least-privilege roles, with encryption at rest.

6. Task 4: Technical Justification

This section explains how the maps and matrices demonstrate CO1. Each of the five questions in the brief is answered in turn.

1. Effectiveness of the concept map and the strongest design decisions

The maps are effective because they are layered and labelled: every arrow reads as a sentence, so a reader can verify each claim. Four design decisions matter most. First, a single anchor node per map with a clear top-down hierarchy keeps the structure readable. Second, consistent colour coding separates service models, characteristics, API concepts and user responsibility at a glance. Third, cross-links (for example virtualization enables elasticity, REST delivers on-demand access) show that cloud concepts form a connected system rather than a list. Fourth, the master map links the abstract concepts to the CloudLearn project, which proves they are applied and not merely recalled. Together these map directly to the rubric criteria of concept identification, relationship mapping and hierarchy.

2. Influence of scalability, elasticity and resource management; what would go wrong without them

Scalability shapes the architecture into stateless API instances behind a load balancer, read replicas and serverless functions. Elasticity adds the monitoring-driven loop that grows and shrinks capacity automatically, and resource pooling, delivered by virtualization, lets the provider supply that capacity from a shared pool. Without these capabilities the team would have to provision for peak load in advance. In quiet periods that wastes money on idle servers; in busy

periods (for example exam week) a fixed capacity would be exhausted, causing slow responses, errors and outages, and manual intervention would be needed to add servers. Without measured service the cost of each component would also be invisible, so waste could not be found.

3. Justification of the APIs, web services and cloud services selected

A REST API through an API gateway is selected because REST is stateless, lightweight (HTTP with JSON) and works equally for the web and mobile clients; statelessness is what allows any instance to serve any request and therefore makes horizontal scaling possible. SOAP is shown in the maps as an alternative for formal enterprise contracts but is not needed here. The managed identity service, database, object storage and monitoring are chosen because each removes an operational burden and is hard to build securely; the serverless function is chosen for the bursty AI workload. The overall effect is that the team spends effort on learning features and not on servers, which improves efficiency, reliability and time to market. Services that did not add value (for example dedicated virtual machines for the main API) were deliberately left out.

4. Relationships between components: reliability, performance and user experience

Reliability improves because components are decoupled: the notification queue lets the API respond even if email delivery is slow; managed database backups and replicas allow recovery and failover; the load balancer routes around a failed instance. Performance improves because the CDN serves static content near the user, read replicas relieve the database, large files are downloaded straight from object storage, and the heavy AI computation runs in a separate scalable function instead of blocking the API. User experience follows from both: fast page loads, consistent responses under load, and secure single sign-on. Monitoring closes the loop by detecting problems and triggering scaling before users notice.

5. Reflection on developing the map

Building the maps showed that the cloud characteristics are not independent features: virtualization produces resource pooling and elasticity, and web service APIs produce on-demand access, so removing one enabler weakens several characteristics at once. Mapping the service models also made the shared responsibility boundary concrete: choosing a higher-level service reduces operational duties but also reduces control, which is a real trade-off. If redesigning, I would add deployment-model detail (public, private, hybrid) to the architecture, include cost and availability numbers on the scaling points, show failure paths and a multi-zone layout, and add a data-protection view for compliance. I would also validate the design with a small load test to confirm the scaling thresholds.